Task 3 – NumPy Operations

Overview

This task introduces NumPy, Python's core library for numerical computing. Students will work with multi-dimensional arrays, perform mathematical and statistical operations, and explore linear algebra — building the foundation for data analysis and machine learning workflows.

Learning Objectives

- Create and manipulate 1D, 2D, and 3D NumPy arrays
- Apply array indexing, slicing, and reshaping techniques
- Perform element-wise mathematical operations and broadcasting
- Use statistical functions: mean, median, std, and percentiles
- Carry out linear algebra operations: dot product and matrix multiplication
- Generate random numbers and conduct sampling

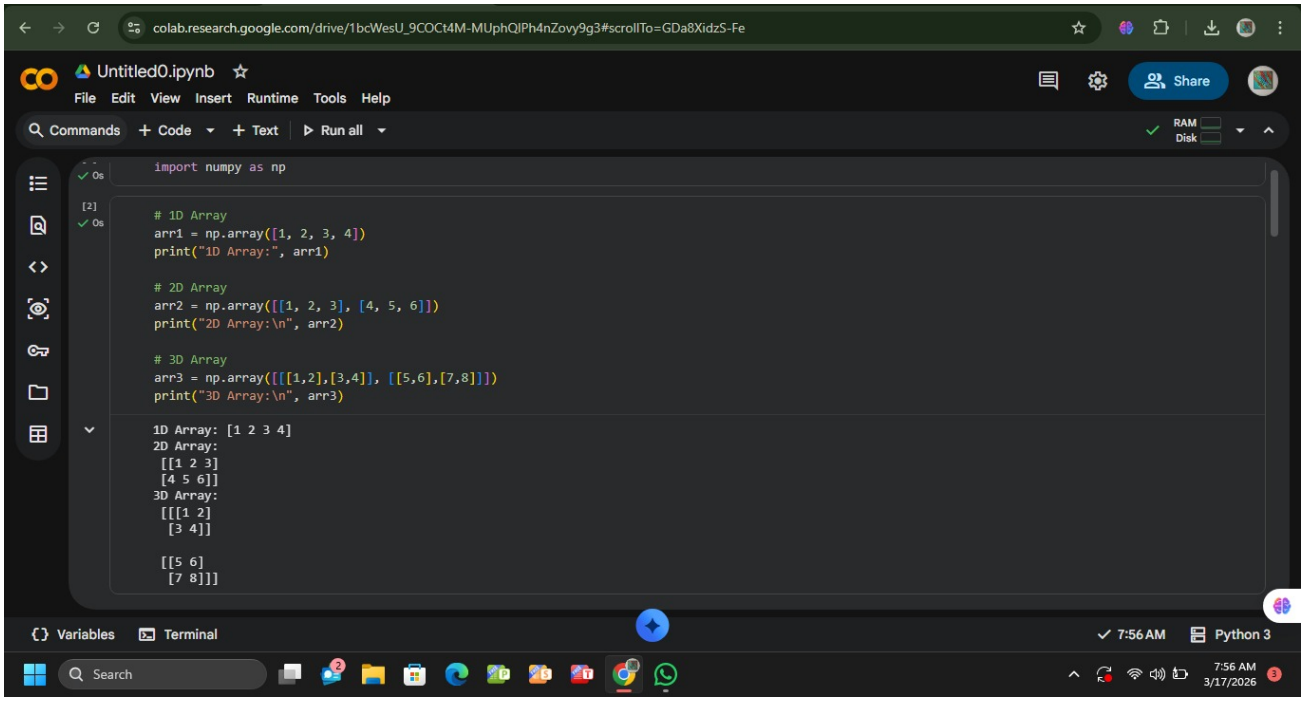
Dataset

Stock prices or temperature measurements — real-world numerical datasets ideal for demonstrating array operations, statistical summaries, and data transformations.

Deliverable

A Jupyter Notebook demonstrating all NumPy operations with clear code examples, inline output, and brief explanations for each concept covered.

Preview:



[3]

```python
arr = np.array([10, 20, 30, 40, 50])

# Indexing
print("First Element:", arr[0])

# Slicing
print("Slice:", arr[1:4])

# Reshaping
arr_reshape = np.array([1,2,3,4,5,6])
reshaped = arr_reshape.reshape(2,3)

print("Reshaped Array:\n", reshaped)
```

```
First Element: 10
Slice: [20 30 40]
Reshaped Array:
 [[1 2 3]
 [4 5 6]]
```

[4]

```python
a = np.array([1,2,3])
b = np.array([4,5,6])

# Element-wise operations
print("Addition:", a + b)
print("Multiplication:", a * b)

# Broadcasting
c = np.array([10])
print("Broadcasting:", a + c)
```

```
Addition: [5 7 9]
Multiplication: [ 4 10 18]
Broadcasting: [11 12 13]
```

[5]

```python
data = np.array([10, 20, 30, 40, 50])

print("Mean:", np.mean(data))
print("Median:", np.median(data))
print("Standard Deviation:", np.std(data))
print("Percentile (25%):", np.percentile(data, 25))
```

```
Mean: 30.0
Median: 30.0
Standard Deviation: 14.142135623730951
Percentile (25%): 20.0
```

[6]

```python
A = np.array([[1,2], [3,4]])
B = np.array([[5,6], [7,8]])

# Dot Product
print("Dot Product:\n", np.dot(A, B))

# Matrix Multiplication
print("Matrix Multiplication:\n", A @ B)
```

```
Dot Product:
 [[19 22]
 [43 50]]
Matrix Multiplication:
 [[19 22]
 [43 50]]
```

[8]

```python
# Example stock prices
prices = np.array([100, 102, 101, 105, 110, 108])

# Daily returns
returns = np.diff(prices)

print("Prices:", prices)
print("Daily Changes:", returns)

print("Average Price:", np.mean(prices))
print("Max Price:", np.max(prices))
print("Min Price:", np.min(prices))
```

```
Prices: [100 102 101 105 110 108]
Daily Changes: [ 2 -1  4  5 -2]
Average Price: 104.33333333333333
Max Price: 110
Min Price: 100
```

[8]

Start coding or generate with AI.